REMORA: Real-time Expressive Motion to Output Routing Audio

Timothée Dravet
Polytech Marseille, Aix-Marseille Université
France
timothee.dravet@etu.univ-amu.fr

Daniel Overholt
CREATE, Aalborg University
Denmark
dano@create.aau.dk

Abstract

REMORA turns a martial arts staff (bô) into a musical instrument: spins, tilts, and strikes become sound in real time. A wireless sensor unit on the staff tracks its motion and sends it to a companion audio plugin, which we built around a simple idea - instead of hard-wiring one fixed gesture-to-sound behaviour, the mapping itself is a small patch of connected building blocks that the performer can see and rewire live, on a visual canvas inside the plugin, much like a modular synthesiser. We show that this same small set of building blocks is enough to reproduce, as editable patches, everything from simple tilt-controlled pitch to compound movement-quality mappings inspired by Laban Movement Analysis. This paper describes the instrument's hardware, its self-calibration method, the mathematics behind turning raw orientation into musical control, and the live patching interface itself, and reflects on what it means to make an instrument's gestural vocabulary an explicit, editable part of the instrument.

CCS Concepts

• **Applied computing** → **Sound and music computing**;
• **Software and its engineering** → *Visual languages.*

Keywords

digital musical instrument, gestural control, inertial sensing, Laban Movement Analysis, OSC, staff instrument

1 Introduction

Motivation and Background

REMORA grew out of the first author's parallel practice in martial arts and music, and a frustration shared by many hand-built digital musical instruments: once a gesture vocabulary is designed and coded, changing it means rebuilding the instrument rather than performing with it. That friction is the paper's real starting point - not which single mapping best suits a staff, but whether a staff instrument's gestural vocabulary could instead be something the performer authors and revises live, the way a modular-synthesis patch is built and rebuilt on a rack.

The bô staff itself sits in a gap NIME has left largely unexplored: a full-body, swung, two-handed controller with its own rotational inertia and reach, distinct from the small handheld or body-mounted objects most inertial-sensor DMIs use. Pairing that underused controller form with a mapping layer that can be reshaped at performance time, rather than fixed at design time, is the combination this paper investigates.

Objectives of the Study

This article supports four claims, each argued based on the system as it currently operates rather than being proposed as future work:

(1) A small dataflow language - source, math, and sink node primitives - is embedded directly in the audio plugin and evaluated once per audio block as a directed acyclic graph (DAG); the performer rewires it live, without recompiling any code.
(2) That same handful of primitives is expressive enough to reproduce eleven previously hand-coded mapping strategies as ordinary presets: the mapping design space explored by hand over the course of the project turns out to be spanned by a fairly compact algebra of building blocks.
(3) A magnetometer-free, world-frame heading estimate ("facing") is derived purely from the calibrated orientation stream, eluding the magnetic interference a stage rig would otherwise cause.
(4) A mount-agnostic calibration transform lets the same instrument logic run correctly regardless of where the sensor is physically mounted on the staff.

Scope and Applications

REMORA targets solo live performance with a single staff and a single IMU - not an installation piece, not a multi-performer controller (at least not yet, see Future Improvements). Worth being upfront about what's out of scope for this paper too: no formal user study, no controlled comparison against other staff instruments, and no claim that patching a mapping is easier to learn than a fixed one - only that the capability exists and is expressive.

2 Related Work

Staff, Rod, and Baton DMIs

I need to place REMORA against prior staff/rod-shaped DMIs - the T-Stick, conducting batons, poi/staff-flow instruments - covering how each senses motion and, more importantly, what vocabulary of gesture-to-sound mapping each exposes to the performer, since that vocabulary is the axis REMORA differs on most.

Inertial Sensing and Visual Patching in NIME

Two threads to cover here: prior IMU-based DMIs and how they handle calibration/orientation extraction, and - separately - visual dataflow patching environments (Max/MSP, Pure Data, VCV Rack) as the closest existing point of comparison for the node-graph interface. I should also fold in prior use of Laban Effort qualities as a mapping source in NIME work.

3 Design and System Architecture

Initial Design Requirements

Modelled on how a proper requirements list should read, not just a feature dump:

- *Full-body, swingable staff form factor - not a handheld controller*
- *Wireless sensing during performance; a cable only for charging*
- *Live-editable mapping, with no audio interruption when switching or rewiring*
- *Mount-agnostic calibration, independent of the sensor's physical placement on the staff*
- *Low enough end-to-end latency for the staff to feel driven, not laggy*

Architecture Overview

Here's where I lay out the two-part architecture: firmware-side sensing and OSC (Open Sound Control) transport on one end, plugin-side reception, calibration, graph evaluation, and synthesis on the other. The signal-flow diagram belongs in this section.

4 Implementation

The Build

This is where I go through the physical build: the staff-mounted enclosure, the parametric OpenSCAD casing, and how the electronics are secured to the staff without changing how it's normally gripped or swung.

Hardware

This is where I go through the ESP32-S2 + MPU-9250 sensor package, the 100 Hz I2C (Inter-Integrated Circuit) sampling loop, and the Wi-Fi/UDP (User Datagram Protocol)/OSC packet format the firmware streams to the host.

Software: Calibration and Motion Mathematics

Every equation below comes straight from the running code, each one is tagged with the function it lives in.

Orientation and Calibration. **Rotation.** Orientation is tracked as a unit quaternion (no gimbal lock); rotating v by q gives $v' = q\, v\, q^{-1}$, with q^{-1} the conjugate. Euler angles are derived only once, at the end, for the handful of mappings that still need them.

[MathHelpers::rotate] [conjugate] [rotateVectorByQuaternion] [toEuler] [multiply]

Calibration, stage 1 - which axis is "forward". The sensor can be glued onto the staff at any orientation, so each candidate axis $L \in \{\pm X, \pm Y, \pm Z\}$ is scored against three reference poses A, B, C (quaternions q_A, q_B, q_C, chosen so forward varies substantially between them while up/right do not):

$$v_P = \widehat{\text{rotate}(L, q_P)}\,, \qquad e(L) = |v_A \cdot v_B| + |v_B \cdot v_C| + |v_C \cdot v_A|$$

A low $e(L)$ (near-orthogonal v_A, v_B, v_C) marks the genuine forward axis; restricted to right-handed candidates,

$$L^* = \underset{L\,:\,(v_A \times v_B)\cdot v_C < 0}{\arg\min} \; e(L)$$

gives the alignment quaternion q_{align}.

[computeAlignQuat]

Calibration, stage 2 - fixing the twist. The sensor could still be rotated around L^*. The measured forward/up/right triad and the ideal instrument-frame triad are each orthonormalized via Gram-Schmidt into U and W; the aligning rotation is the orthogonal-Procrustes closed form

$$R = W U^{\top},$$

converted to a quaternion q_{corr} via Shepperd's method.

[computeCorrection] [MathHelpers::fromMatrix]

Runtime. Every live reading becomes instrument space via $q' = q_{\text{corr}}\,(q_{\text{raw}}\, q_{\text{align}})$ - independent of where the sensor sits on the staff.

[REMORAProcessor::getCalibratedQuat]

Motion Feature Extraction. Everything below runs once per received packet, inside the shared analyzer that every mapping graph reads from - so no individual mapping repeats this work. The smoothing rates, thresholds, and envelope times named throughout this subsection were tuned by ear against the physical instrument during iterative testing, not derived analytically or fit to recorded data; see Discussion for the resulting lack of a perceptual evaluation of these choices.

[StaffMotionAnalyzer::computeFrame]

Table 1: Per-packet motion features.

Feature	Purpose
Timestep Δt	reject implausible packet gaps
Tip vector / rotation axis	derive tip velocity & spin axis
Gravity / dynamic accel.	separate gravity from motion
Laban Weight	movement forcefulness
Laban Time	suddenness of rotation-speed change
Laban Space	spin-axis consistency (focus)
Laban Flow	bound vs. free movement
Spin classification	vertical/horizontal spin & direction
Facing	east-west vs. north-south heading
Continuous spin count	count full turns per spin
Axial thrust	detect stab/strike along staff axis

Timestep. Δt is the time between two packets, clamped to a 0.1-200 ms valid range so a dropped or duplicated packet can't send the integrators below flying.

[calculateDeltaTime]

Tip vector and rotation axis. The tip direction $p_n = \text{rotate}((1,0,0), q_n)$ is buffered over the last three samples to derive tip velocity and, from that, the axis the staff is currently spinning around.

[updateTipPositionHistory] [calculateRotationAxisAtMidpoint]

Gravity and dynamic acceleration. The accelerometer always mixes gravity in with whatever motion the performer is making. To separate the two, REMORA keeps a running estimate of which way "down" currently is, by smoothing the inverse-rotated world $+Z$ axis:

$$g_n = \text{onePole}\big(g_{n-1},\ \text{rotate}(+Z,\ q_n^{-1}),\ \alpha{=}0.10\big)$$

Subtracting this gravity estimate from the raw reading leaves just the motion: $a_{\text{dyn}} = a_{\text{raw}} - g$.

[updateGravityVector] [calculateDynamicAcceleration]

Laban features. All four Laban Effort qualities reduce to the same shared one-pole smoother,

$$y_n = y_{n-1} + \alpha\,(x_n - y_{n-1}), \qquad \alpha \in (0, 1],$$

applied to a derivative of either dynamic acceleration or gyroscope magnitude: Weight integrates dynamic acceleration into a leaky velocity estimate (its magnitude, fast-attack/slow-release enveloped) to capture forcefulness; Time is the clamped-positive rate of change of *raw* gyroscope magnitude, capturing suddenness; Space is the smoothed absolute dot product between consecutive rotation-axis directions, capturing how consistently the staff spins around the same axis versus wandering; Flow is the smoothed, clamped jerk of dynamic-acceleration magnitude, split into `flow_bound` (controlled) and $1 -$ `flow_bound` (free).

[`MathHelpers::applyOnePoleFilter`]
[`integrateVelocityForLabanWeight`] [`updateLabanWeight`]
[`updateLabanTime`] [`updateLabanSpace`] [`updateLabanFlow`]

Spin classification. Is the staff spinning like a baton (vertical) or more like a fan blade (horizontal)? Let $(\text{axis}_x, \text{axis}_y, \text{axis}_z)$ be the α=0.35 one-pole-smoothed rotation axis from above. The spin is classified vertical when

$$\sqrt{\text{axis}_x^2 + \text{axis}_y^2} \;\geqslant\; |\text{axis}_z|,$$

i.e. when the horizontal-plane component outweighs the z component. Spin direction (CW/CCW) then comes either from the sign of the dominant in-plane axis component, or - in the Bozendo mappings - from the sign of that axis's dot product against a reference heading latched once, the first time a qualifying vertical spin happens after the mapping loads.

[`updateSpinClassificationByAbsoluteComponent`]
[`updateSpinClassificationByReferenceAzimuth`]

Facing. Rather than compute a compass heading - which would need a magnetometer, and stages are full of magnetic interference - REMORA asks a simpler question: is the staff pointing more east-west or more north-south? Let $(\text{dir}_x, \text{dir}_y)$ be the horizontal-plane components of whichever direction is currently relevant - the rotation axis during vertical spins, the tip direction during horizontal ones. Facing is then just a hysteresis comparison,

$$|\text{dir}_x| \;\geqslant\; 1.15 \cdot |\text{dir}_y|,$$

where the 1.15 factor gives asymmetric switch points ($\approx 48°/42°$) so the classification doesn't flicker right at the boundary.

[`updateFacingClassification`]

Continuous spin count. Full turns in the current spin, from $\|\omega\| \cdot \Delta t$ accumulated to 360° per lap; resets whenever the spin classification changes.

[`accumulateContinuousSpins`]

Axial thrust detection. Flags a stab/strike when the jerk of the tip-projected dynamic acceleration crosses zero as its magnitude passes 1.5g, gated against fast spins ($\|\omega\| < 90°/\text{s}$) and cooled down between hits.

[`detectAxialThrustPeaks`]

Software: Mapping Architecture

Here I explain the `NodeGraph`/`NodeTypeRegistry` *design: three node categories (Source - raw IMU channels and derived motion/Laban features; Math - arithmetic, shaping, dynamics, lookup tables; Sink - synth parameters, from scalars like root frequency to indexed arrays like per-voice gain/pan), each a small typed function registered once. A graph is just nodes plus typed input-to-output wiring, serialised to/from XML (Extensible Markup Language) as a preset - and I want to be clear that* `GraphMappingStrategy` *makes the graph itself the only mapping type the synth ever talks to; there's no separate hand-written mapping path anymore.*

Graph Evaluation and Primitive Library. **DAG evaluation.** A patch is a directed graph of typed nodes; `NodeGraph::connect` provisionally adds an edge and recomputes a topological order with Kahn's algorithm (in-degree queue) via `NodeGraph::recomputeTopoOrder` - if the graph does not fully drain (a node's in-degree never reaches zero), the edge closed a cycle and is reverted. Each audio block, `NodeGraph::evaluate` walks the cached order once, and every node computes

$$y_i = f_{\text{type}(n_i)}\left(x_{i,1}, \ldots, x_{i,m}\,;\,\theta_i\right)$$

where n_i is the i-th node in topological order, $\text{type}(n_i)$ its registered node-type function, $x_{i,1}, \ldots, x_{i,m}$ its wired input values, θ_i its parameters, and y_i its resulting output(s) - so the whole patch is one composed function of the raw sensor frame, evaluated once per block with no re-sorting on the audio thread.

Primitive vocabulary. Every node type is a single small pure function (or, for the few stateful ones, a function plus one scratch struct), registered once: arithmetic (add / subtract / multiply / divide / abs / max / min / equals), shaping (mapRange, clamp, threshold, crossfade, quantizeSteps, sine, atan2, semitonesToHz), lookup tables (a general 1-D table and a 3-boolean-selector table - the same node type that implements the Azimut root table below), and stateful dynamics (variable-rate one-pole smoother, leaky integrator, retrigger envelope, hysteresis step, derivative, latched smoother).

[`Graph::buildAllNodes`]

Software: Named Mappings as Graph Presets

I'd rather show the generalisation claim than just assert it, so this subsection walks through how Simple (tilt-to-pitch), Bowed Chord, Lead+Drone, Spin Filter, the Bozendo V1/V2 Laban-driven mappings, and the Azimut family (facing-based eight-way root table, spin-count filter sweep, axial thrust transients, plus the Azimut+/Reverb/Ben's/Spin Voices variants) are each just a specific wiring of the same node vocabulary - built once in C++ via `GraphBuilder`*, and now editable as ordinary patches.*

Azimut-Specific Transformations. Shared by every Azimut-family preset.

[`Graph::Presets::buildAzimutCore`]

Root-note table. The spin-plane/direction/facing root-note table above is not an if/else chain - it is literally an 8-entry lookup table indexed by three booleans (`isVertical`, `isCCW`, `isFacingEast`) through the graph's generic 3-selector LUT node.

Root morph. The target semitone value is chased by a variable-rate one-pole filter whose rate is

$$\text{rate} = \max\left(\text{clamp}\left(\tfrac{\|\omega\|}{2000},\ 0.005,\ 0.2\right),\ \ 0.5 \cdot \text{onsetEnv}\right)$$

where `onsetEnv` is a retrigger envelope (decay 0.90) that fires the instant the staff leaves rest - faster motion, or a fresh onset, morphs the root pitch in quicker.

Filter sweep. The spin count drives a phase modulator whose sine output is remapped onto the filter's cutoff range and smoothed:

$$c_{\text{raw}} = \text{mapRange}\left(\sin(1.5 \cdot \text{spinCount}),\ [-1, 1],\ [400, 20000]\text{Hz}\right)$$
$$\text{cutoff} = \text{onePole}\left(c_{\text{raw}},\ \text{rate}{=}0.03\right)$$

where spinCount is the unsigned lap counter from Continuous Spin Count above, mapRange linearly rescales its argument from the first interval to the second, c_{raw} the resulting unsmoothed cutoff target, and onePole the smoothing primitive defined earlier.

[`buildSpinCountLpfHz`]

Graphical Interface: Live Patching as Performance Practice

This is probably the subsection I want to spend the most space on after the mathematics. I want to present the node-graph editor as the instrument's primary authoring surface, not a debug tool: a pannable, zoomable canvas on which each node renders as a coloured card - green for Source, yellow for Sink, neutral for Math - labelled with its category and inline, live-editable parameter fields; typed input/output pins along each card's edges accept click-and-drag connections rendered as cubic-Bézier wires, colour-highlighted while dragging and hit-testable afterward for rewiring or deletion via context menu. New nodes are added from a categorised picker at any canvas position; an auto-layout pass untangles a freshly loaded preset into left-to-right source-to-sink columns. A dirty-state indicator tracks whether the active patch has diverged from its saved preset, with one-click revert - so a performer can improvise a mapping change mid-rehearsal and either keep or discard it, treating the mapping itself as something authored and iterated on at the same level as the sound it produces.

Software: Synthesis Engine

Still need to write up `BoStaffSynth` *here - the shared four-voice additive engine and its mapping-agnostic parameter surface (gain, drive, noise, filter, pan, reverb send) that every graph's sink nodes write into.*

Additive Synthesis Engine.

[`BoStaffSynth::processBlock`] [`SynthVoice::tick`]

Harmonic bank. Each of the 4 voices sums 6 phase-accumulated sine partials at integer multiples of a (vibrato- and chorus-modulated) fundamental, each with its own independently smoothed target amplitude.

Waveshaping. A rational soft-clip saturator, applied per-voice when `driveAmt` > 0:

$$y = \frac{x\,(1 + \text{driveAmt})}{1 + \left|x\,(1 + \text{driveAmt})\right|}$$

where x is the voice's summed harmonic-bank sample before shaping and `driveAmt` the mapping-supplied drive amount.

Resonator. A short feedback comb (1800-sample delay, 0.35 feedback, mixed 0.7 dry / 0.3 wet) is layered onto every voice for a string-like coloration.

Modulation. Vibrato and tremolo are sinusoidal multipliers on pitch and gain respectively:

$$f' = f\left(1{+}\text{depth}{\cdot}\sin\phi_v\right) \qquad g' = g\left(1{-}\text{depth}{\cdot}(0.5{+}0.5\sin\phi_t)\right)$$

where f, g are the fundamental frequency and master gain before modulation, f', g' the modulated values actually used, and ϕ_v, ϕ_t the vibrato and tremolo LFO phase accumulators (radians), each advancing by its own rate every sample. A per-voice phase-offset sine (depth 0.003, rate 0.3 Hz) adds chorus detuning; chord changes cross-fade linearly between old and new semitone offsets over 200 ms rather than jumping.

Noise and filtering. An xorshift32 PRNG feeds a one-pole lowpass (coefficient `noiseLpCoef`) before being mixed in at `noiseAmount`; the summed stereo output passes through a JUCE state-variable TPT lowpass, its cutoff set directly each block from the graph's `sink.lpfCutoffHz` (only range-clamped, not ramped), and an optional Schroeder-style `juce::dsp::Reverb` send whose wet level *is* ramped through a per-sample `SmoothedValue` to avoid zippering when the active mapping changes.

Pitch/frequency conversion.

$$f = f_{\text{root}} \cdot 2^{\,\text{semitones}/12}$$

where f_{root} is the mapping's current root frequency in Hz and semitones the signed offset applied to it, giving the equal-tempered result f.

[`MathHelpers::semitoneRatio`] [`convertSemitonesToHertz`]

5 Development Iterations

Modelled on the reference paper's iteration log rather than presenting the final architecture as if it arrived fully formed. Mine reads roughly: (1) firmware + OSC transport + a single hardcoded tilt-to-pitch mapping, just to prove the sensor-to-sound pipeline worked at all; (2) hand-building the full progression of named mappings - Bowed Chord through Bozendo to Azimut and its variants - each one a bespoke C++ class; (3) noticing the hand-built mappings were all reducible to the same handful of primitives, and generalising that into the node-graph engine so every prior mapping became a preset instead of a class.

Iteration 1: Sensing Pipeline

Firmware reading the IMU and streaming OSC, a minimal receiver on the plugin side, and one hardcoded tilt-to-pitch mapping - the goal here was purely proving the pipeline end to end, not expressivity.

Iteration 2: Hand-Built Mapping Library

The eleven named mappings built one at a time as their own C++ classes - Simple, Bowed Chord, Lead+Drone, Spin Filter, Bozendo V1/V2, Azimut and its variants, Ben's, Spin Voices - each adding a new gesture vocabulary (chords, Laban Effort, world-frame facing, transient detection) on top of the shared `StaffMotionAnalyzer`.

Iteration 3: Generalising into the Node Graph

Recognising that every hand-built mapping was really just arithmetic, shaping, and a few stateful primitives wired together in a fixed pattern - and rebuilding the whole mapping layer as a small generic graph engine, with the eleven prior mappings ported over as presets rather than deleted.

6 Evaluation

No formal study yet - I want to be honest about that rather than dress up informal playtesting as more than it is.

Test Session with a Performer

TBD - modelled on the reference paper's semi-structured test session: get someone who actually performs with staff-like objects (martial arts, poi, staff spinning) to try REMORA, and take notes rather than assume my own playtesting generalises.

Notes and Suggestions. TBD once the session happens.

In Summary. TBD once the session happens.

7 Discussion

Advantages and Limitations

I want to be honest here about what the generalisation into a graph actually buys versus what it costs: expressiveness and shareability of mappings, weighed against the added cognitive step of patching for performers used to fixed instruments. Worth folding in the informal playtesting observations, and I should be upfront about the current limitations - single-IMU ambiguity, no formal user study yet, no perceptual evaluation of the facing/calibration accuracy.

Future Improvements

Extensions I keep coming back to: gesture-triggered mode switching, a merged rest/motion mapping, a two-IMU dual-tip configuration, further Laban Effort mapping targets, ML-based discrete gesture recognition, and eventually exposing the node graph itself as a shareable/community patch format.

8 Conclusions

I want to close by making REMORA's real novelty explicit: it's architectural, not just one more instrument - collapsing eleven hand-built gesture mappings into presets of one small, live-patchable dataflow language embedded in the plugin, grounded in an explicit mathematical account of calibration, orientation, and movement-quality extraction.

Acknowledgments

This work was carried out during the first author's internship at CREATE, Aalborg University, Denmark.

9 Ethical Statement

Needs real content before submission, but the shape should cover: accessibility (the staff form factor assumes a certain range of mobility and two-handed grip strength - name that as a limitation rather than leaving it unstated); environmental impact of the staff-mounted hardware build (battery, 3D-printed/laser-cut enclosure materials, whether they're recyclable); socio-economic fairness (REMORA is built entirely on FLOSS/FLOSH tools - JUCE, PlatformIO, OpenSCAD - worth stating plainly, since that's a genuine point in its favour under the NIME Code of Practice); no human-participant data collected beyond the informal test session in the Evaluation section, with informed consent obtained for that session; and disclosure of any AI assistance used in drafting this manuscript, per the NIME Code of Practice on Ethical Research - separate from the instrument's code, which was written entirely by hand.

Temporary page!

LaTeX was unable to guess the total number of pages correctly. As there was some unprocessed data that should have been added to the final page this extra page has been added to receive it.

If you rerun the document (without altering it) this surplus page will go away, because LaTeX now knows how many pages to expect for this document.